UNIVERSITAT DE BARCELONA
Facultat de Matemàtiques i Informàtica

**GRAU DE MATEMÀTIQUES**

**Treball final de grau**

# GRAPH NEURAL NETWORKS FOR AUTONOMOUS THEOREM PROVING

**Autor: Carlos Fernández Lorán**

**Director:** Dr. Carles Casacuberta
**Realitzat a:** Departament of Mathematics and Computer Science

**Barcelona,** March 10, 2026

# Contents

**Introduction** **ii**

**1 Generalized Algebraic Theories** **1**

1.1 Formal Definition . . . . . 3

1.2 The Substitution Lemma . . . . . 5

1.3 Models and Homomorphisms . . . . . 7

1.4 Contexts and Realizations . . . . . 9

**2 The Proposed Model** **15**

**Bibliography** **17**

## Abstract

The main goal of this work is to prove the existence of.....

**Notation:** We will work over an algebraically closed field $K$ of characteristic zero.

2020 Mathematics Subject Classification. 11G05, 11G10, 14G10

# Chapter 1

# Generalized Algebraic Theories

The goal of this chapter is give meaning to the concept of term, type and theory, while introducing some tools that we will use later when we present the algorithm.

By theories we mean Generalized Algebraic Theories (or GATs), introduced by John Cartmell in his thesis [1] [2]. GATs give us both a very idiomatic way of presenting theories and powerful interpretation tools like Context Categories, introduced later in this section. The notion of generalized algebraic theory is a generalization of the notion of many-sorted algebraic theory in the following manner: while in Martin Löff's many-sorted algebraic theories sorts are constant types, interpreted as sets, in generalized algebraic theories sorts might be constant or variable, over a specified range of types, and variable types are interpreted as families of sets. So a generalized algebraic theory consists of:

1. a set of sorts, that can be constant or variable;
2. a set of operators, with the sort of the arguments and value specified;
3. a set of axioms, where each axiom is an identity between similar expressions.

Let's present category theory for example: The set of sorts contains two elements $Ob$ and $Hom$. $Ob$ is a constant type and $Hom$ is a variable type, for a pair of terms of type $Ob$, name them $x, y$, we have a type $Hom(x, y)$. Here we see a key point in general algebraic theories, types vary over terms, no over types. The set of operators also has two elements $id$ and $\circ$. The $id$ operator has one argument $x$ of type $Ob$, and returns a value of type $Hom(x, x)$. This shows how the type of value of an operator might vary depending on the arguments. The $\circ$ operator (or composition) expects 5 arguments, three of type $Ob$, name them $x, y, z$, one of type $Hom(x, y)$, say $f$, and one of type $Hom(y, z)$, named $g$, and it returns a value of type $Hom(x, z)$. Here we see that the type of an argument might also vary

depending on a previous argument. Informally, instead of writing $\circ(x, y, z, f, g)$ we can write $\circ(f, g)$, as the first three arguments might be inferred from the last two. This notation will be discussed in a further section.

Now we will introduce the notation we will be using when talking about terms and types. This notation will be formally defined in a later section. So consider you have a set of variables $V$, we will associate terms to variables as required. When asserting that a term $t$ is of type $\Delta$ we will write $t : \Delta$, but if $t$ has variables $x_1, \ldots, x_n$ occurring in it this assertion does not make sense unless we assign some types to $x_1, \ldots, x_m$. To do this we will write:

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{t : \Delta}$$

that reads as: if $x_1$ is a variable of type $\Delta_1$, ...and if $x_n$ is a variable of type $\Delta_n$, then $t$ is a term of type $\Delta$. Similarly, if we want to assert that if $x_1$ is a variable of type $\Delta_1$, ...and if $x_n$ is a variable of type $\Delta_n$, then $\Delta$ is a type, we can write:

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{t : \Delta}$$

this assertions are called rules. Rules are used to express which expressions are well-formed. We will work with this rules as units instead of the base expressions. This is because terms and types do not make sense by themselves but they need to be thought of in a context.

Axioms are also written as rules. The conclusion of those rules are equalities between types or between terms of the same type. We write those rules as:

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta = \Delta'} \qquad \frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{t = t' : \Delta}$$

and read as: if $x_1$ is a variable of type $\Delta_1$, ...and $x_n$ is a variable of type $\Delta_n$, then $\Delta = \Delta'$, or $t = t'$ as terms of type $\Delta$, respectively.

To continue with the previous example, we can write the axioms of category theory as:

$$\begin{aligned}
&x, y : Ob, f : Hom(x, y), \quad \circ(id(x), f) = f \\
&x, y : Ob, f : Hom(x, y), \quad \circ(f, id(y)) = f \\
&w, x, y, z : Ob, f : Hom(w, x), g : Hom(x, y), h : Hom(y, z), \\
&\qquad \circ(\circ(f, g), h) = \circ(f, \circ(g, h))
\end{aligned}$$

When presenting a theory's language i.e. the sets of sorts and operators, we must present each symbol with a rule specifying the role of this symbol, either as a sort symbol or as specifically typed operator symbol. This rules are called introductory rules. In the case of a sort symbol $A$ the rule should be like

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{A(x_1, \ldots, x_n) \text{ is a type}}$$

clearly specifying the types $A$ is dependent on. In the case of an operator symbol, a rule of the form

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{f(x_1, \ldots, x_n) : \Delta}$$

suffices to specify the types of the arguments and the value of $f$.

Finally, to present a theory we will give a set os symbols, each associated to its introductory rule, and a set of axioms. And, of course, everything must be well-formed, something we will address in the following section. So now we can give a formal presentation of the theory of categories:

| *Symbol* | *Introductory Rule* |
|---|---|
| Ob | Ob is a type. |
| Hom | $x, y \in \mathrm{Ob} : \mathrm{Hom}(x, y)$ is a type. |
| $\circ$ | $x, y, z \in \mathrm{Ob}$, $f \in \mathrm{Hom}(x, y)$, $g \in \mathrm{Hom}(y, z) : \circ(f, g) \in \mathrm{Hom}(x, z)$. |
| id | $x \in \mathrm{Ob} : \mathrm{id}(x) \in \mathrm{Hom}(x, x)$. |

*Axioms*

| | |
|---|---|
| $\circ(\mathrm{id}(x), f) = f$, | whenever $x, y \in \mathrm{Ob}$ and $f \in \mathrm{Hom}(x, y)$. |
| $\circ(f, \mathrm{id}(y)) = f$, | whenever $x, y \in \mathrm{Ob}$ and $f \in \mathrm{Hom}(x, y)$. |
| $\circ(\circ(f, g), h) = \circ(f, \circ(g, h))$, | whenever $w, x, y, z \in \mathrm{Ob}$, $f \in \mathrm{Hom}(w, x)$, $g \in \mathrm{Hom}(x, y)$ and $h \in \mathrm{Hom}(y, z)$. |

## 1.1 Formal Definition

In this section we will formally define all of the concepts we just introduced. To give a notion of well-formed introductory rule, we need the concept of drivability, which also depends on the introductory rules. To tackle this problem we will leave aside the question of well-formedness until we have the complete set of derived rules of a theory. We will first define a pretheory and then define a theory as a well-formed pretheory.

We will assume to have countable set of variables $V$. Lets define the notion of rule on an alphabet $W$, the elements of $W$ are called symbols.

**Definition 1.1.** *The set of* ***expressions*** *is defined inductively such that expressions are finite sequences of elements of $W \cup V \cup \{(\} \cup \{)\} \cup \{,\}$ defined by the clauses:*

1. *if $x \in V$, then $x$ is an expression;*
2. *if $L \in W$, then $L$ is an expression;*
3. *if $L \in W$ and $e_1, \ldots, e_n$ are expressions, then $L(e_1, \ldots, e_n)$ is an expression.*

Now we can define the building blocks of a rule:

**Definition 1.2.** *A **premise** is defined to be a sequence of $V\times$ the set of expressions. The empty sequence is allowed and we call it an empty premise. If $((x_1, \Delta_1), \ldots, (x_n, \Delta_n))$ is a premise, we write it as $x_1 : \Delta_1, \ldots, x_n : \Delta_n$.*

**Definition 1.3.** *We call a **conclusion** to one of the following:*

1. *T-conclusion: defined by a single expression $\Delta$ and written as '$\Delta$ is a type'.*
2. *$\in$-conclusion: defined by a pair of expressions $(t, \Delta)$ and written as '$t \in \Delta$'.*
3. *$T$ =conclusion: defined by a pair of expressions $(\Delta_1, \Delta_2)$ and written as '$\Delta_1 = \Delta_2$'.*
4. *$\in$=conclusion: defined by a triple of expressions $(t_1, t_2, \Delta)$ and written as '$t_1 = t_2 : \Delta$'*

Finally a **rule** is determined by a premise $P$ and a conclusion $C$, and is written as $\frac{P}{C}$. We name the rule as $T$-rule, $\in$-rule, $T$ =rule or $\in$=rule according to the form of the conclusion.

Now we can define pretheory and later theory.

**Definition 1.4.** *A **pretheory** consists of*

1. *a set of sort symbols $S$;*
2. *a set of operator symbols $Z$;*
3. *for each sort symbol $A$, an associated rule of the alphabet $S \cup Z$:*

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{A(x_1, \ldots, A_n) \text{ is a type}}$$

   *for some $n \geq 0$. It is called the introductory rule of $A$;*

4. *for each operator symbol $F$, an associated rule of the alphabet $S \cup Z$:*

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{F(x_1, \ldots, x_n) : \Delta}$$

   *for some $n \geq 0$. It is called the introductory rule of $F$;*

5. *a set of axioms. Each axiom is either a $T$ =rule or a $\in$=rule of the alphabet $S \cup Z$.*

Now we will tackle the problem of well-formedness of rules.

**Definition 1.5.** *If $U$ is a pretheory, then*

1. *a* ***context*** *is a premise* $x_1 : \Delta_1, \ldots, x_n : \Delta_n$ *such that the rule*

$$\frac{x_1 : \Delta_1, \ldots, x_{n-1} : \Delta_{n-1}}{\Delta_n \text{ is a type}}$$

*is a derived rule such that* $x_n$ *is distinct from all of* $x_1, \ldots, x_n$.

2. (a) *the rule* $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta \text{ is a type}}$ *is* ***well-formed*** *if the premise* $x_1 : \Delta_1, \ldots, x_n : \Delta_n$ *is a context.*

   (b) *the rule* $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{t : \Delta}$ *is* ***well-formed*** *if* $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta \text{ is a type}}$ *is a derived rule of* $U$.

   (c) *the rule* $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta = \Delta'}$ *is* ***well-formed*** *if* $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta \text{ is a type}}$ *and* $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta' \text{ is a type}}$ *are derived rules of* $U$.

   (d) *the rule* $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{t = t' : \Delta}$ *is* ***well-formed*** *if* $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{t : \Delta}$ *and* $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{t' : \Delta}$ *are derived rules of* $U$.

**Definition 1.6.** *The set of* ***derived rules*** *of a pretheory* $U$ *is the set of rules derivable by the following principles of derivation:*

**Definition 1.7.** *A pretheory is well-formed if all of its introductory rules are well-formed. A* ***theory*** *is a well-formed pretheory.*

## 1.2 The Substitution Lemma

Given $\Delta, t_1, \ldots, t_n$ expressions and $x_1, \ldots, x_n$ variables, then we define the expression $\Delta[t_1|x_1, \ldots, t_n|x_n]$ the result of simultaneous replacement of $x_1, \ldots, x_n$ in $\Delta$ by $t_1, \ldots, t_n$. We call this the Substitution operation.

The substitution lemma states that the set of derived rules of a theory is closed under the operation of substitution of correctly typed terms for variables. We could add substitution as one of the derivation rules but would make applying induction very messy.

**Definition 1.8.** *A derived rule of* $\dfrac{y_1|\Omega_1, \ldots, y_n|\Omega_n}{Conclusion}$ *of a theory* $U$ *is said to have the* ***substitution property*** *if for all contexts* $Q$ *of* $U$*, for all expressions* $s_1, \ldots, s_m$ *of* $U$ *if the rules*

$$\frac{Q}{s_1 : \Omega_1}, \frac{Q}{s_2 : \Omega_2[s_1|y_1]}, \ldots, \frac{Q}{s_m : \Omega_m[s_1|y_1, \ldots, s_{m-1}|y_{m-1}]}$$

*are all derived rules of $U$, then the rule*

$$\frac{Q}{Conclusion[s_1|y_1, \ldots, s_m|y_m]}$$

*is a derived rule of $U$.*

We will see that all derived rules hold this property.

**Lemma 1.9.** *If* $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{Conclusion}$ *is a derived rule of $U$, then any variable appearing in the conclusion occurs among $\{x_1, \ldots, x_n\}$.*

*Proof.* Easily obtained by applying induction over the derivation rules. □

**Lemma 1.10.**

1. *the premise of a derived rule is a context,*
2. *if $x_1 : \Delta_1, \ldots, x_n : \Delta_n$ is a context, then for all $1 \leq i \leq n$, then* $\dfrac{x_1 : \Delta_1, \ldots, x_{i-1} : \Delta_{i-1}}{\Delta_i \text{ is a type}}$ *is a derived rule.*

*Proof.* □

**Lemma 1.11** (The Substitution Lemma)**.** *Every derived rule of a theory $U$ has the substitution property.*

*Proof.* □

**Corollary 1.12** (Change of Variables)**.** *If* $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{Conclusion}$ *is a derived rule of $U$ and if $y_1, \ldots, y_n$ is a sequence of distinct variables, then*

$$\frac{y_1 : \Delta_1, y_2 : \Delta_2[y_1|x_1], \ldots, y_n : \Delta_n[y_1|x_1, \ldots, y_{n-1}|x_{n-1}]}{Conclusion[y_1|x_1, \ldots, y_n|x_n]}$$

*is a derived rule of $U$.*

## 1.3 Models and Homomorphisms

In this section we will consider the semantics of a theory. To do this we will introduce the notion of model model homomorphism.

First, we will set some notation and basic concepts. We will consider a **tree** to be a tree with a unique least element, in the case that our tree does not have one we will add it. This element will be denoted as 1. Let $\Theta$ be a tree. To denote that $A$ is a node of $\Theta$ we will write $A \in \Theta$, confusing the tree with its set of nodes. To assert that $B$ is a node of $\Theta$ and $B$ succeeds $A$, we write $A \lhd B$ in $\Theta$. Also, if $A$ is a node of $\Theta$ and $A \neq 1$, there exists a unique $n \geq 0$ and $A_1, \ldots, A_n$ such that $1 \lhd A_1 \lhd \cdots \lhd A_n \lhd A$ in $\Theta$.

The set of contexts of a theory $U$ is structured as a tree. Where its least element is the empty context and for any $n \geq 1$ the predecessor of $\langle x_1 : \Delta_1, \ldots, x_n : \Delta_n \rangle$ is $\langle x_1 : \Delta_1, \ldots, x_{n-1} : \Delta_{n-1} \rangle$.

We want to consider the tree of sets, families of sets, families of families of sets, ... Consider the following notation for families: given $A$ is a set, if for every element $a \in A$ $B(a)$ is a set, then the corresponding $A$-indexed family of sets is denoted as $\lambda a \in A.B(a)$ or as $\lambda a.B(a)$; If we have a set $C(a,b)$ for every $a \in A$ and $b \in B(a)$, then $\lambda a \in A.\lambda b \in B(a).C(a,b)$ is an $A$-indexed family of families of sets; and so on. The collection of sets, families of sets, families of families of sets, and so on, is called the **tree of families**. Given $1 \lhd A_1 \lhd A_2 \lhd \cdots \lhd A_n \lhd A$ in the tree of families. Then, for all $a_1 \in A_1$, ..., for all $a_n \in A_n(a_1, \ldots, a_{n-1})$, $A(a_1, \ldots, a_n)$ is a set. Finally we should define the concept of **operator**. If we have $1 \lhd A_1 \lhd \cdots \lhd A_n \lhd A$ in the tree of families. If for all $a_1 \in A_1$, ..., for all $a_n \in A_n(a_1, \ldots, a_{n-1})$, $f(a_1, \ldots, a_n) \in A(a_1, \ldots, a_n)$. We say that $\lambda a_1 \in A_1 \ldots \lambda a_n \in A_n(a_1, \ldots, a_{n-1}).f(a_1, \ldots, a_n)$ is an operator at $A$.

Now we will introduce the notion of a model of a theory. It is important to understand that the derived rules are the ones to be interpreted by the model, not expressions. To denote the interpretation of a rule $R$ by a model $M$ we will use the superscripted $R^M$.

**Definition 1.13.** *Let $U$ be a theory. $M$ is a model of $U$ if*

1. *for each derived T-rule $R$ of the form $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta \text{ is a type}}$, if we consider the rules $R_i$ to be $\dfrac{x_1 : \Delta_1, \ldots, x_{i-1} : \Delta_{i-1}}{\Delta_i \text{ is a type}}$ for all $1 \leq i \leq n$. Then $M$ assigns those rules to $1 \lhd R_1^M \lhd R_2^M \lhd \cdots \lhd R_n^M \lhd R^M$ in the tree of families;*

2. *if $R_t$ is a derived $\in$-rule of the form $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{t : \Delta}$, following the notation of the previous point, $M$ assigns the rule to $R_t^M$ an operator at $R^M$.*

**Definition 1.14.** *If $M$ and $M'$ are models of $U$, $f : M \to M'$ is a* ***homomorphism*** *if*

1. *for every derived $T$-rule $R$ of the form $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta \text{ is a type}}$ there is an operator $f_R$ such that for every $a_1 \in R_1^M, \ldots,$ for every $a_n \in R_n^M(a_1, \ldots, a_{n-1})$ and for every $a \in R^M(a_1, \ldots, a_n)$,*

$$f_R(a_1, \ldots, a_n, R_t^M(a_1, \ldots, a_n)) \in R^{M'}(f_{R_1}(a_1), \ldots, f_{R_n}(a_1, \ldots, a_n))$$

2. *for all derived $\in$-rules $R_t$ of the form $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{t : \Delta}$, for all $a_1 \in R_1^M, \ldots, a_n \in R_n^M(a_1, \ldots, a_{n-1})$,*

$$f_R(a_1, \ldots, a_n, R_t^M(a_1, \ldots, a_n)) = R_t^{M'}(f_{R_1}(a_1), \ldots, f_{R_n}(a_1, \ldots, a_n))$$

3. *if $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta = \Delta'}$ is a derived rule of $U$, then $f_R = f_{R'}$, where $R$ and $R'$ are the rules $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta \text{ is a type}}$ and $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta' \text{ is a type}}$ respectively.*

To define a homomorphism it is enough to give a family $\lambda A \in S.f_A$ of correctly typed operators, where $S$ is the set of sort symbols of $U$.

The category of models and homomorphisms of a theory $U$ is named $U - alg$.

We want to create an equivalence relation on the derived $T$- and $\in$-rules of a theory $U$ which we call the **equivalence relation of identity of denotation** such that $R \equiv R'$ by this relation if any model should interpret $R$ and $R'$ as identical objects.

**Example 1.15.** Semantically we should not distinguish between the rules

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta \text{ is a type}} \qquad \frac{x'_1 : \Delta_1, \ldots, x'_n : \Delta_n[x'_1|x_1, \ldots, x'_{n-1}|x_{n-1}]}{\Delta[x'_1|x_1, \ldots, x'_n|x_n] \text{ is a type}}$$

that only differ on the choice of the variables.

**Example 1.16.** We should not distinguish either between

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta \text{ is a type}} \qquad \frac{x_1 : \Delta'_1, \ldots, x_n : \Delta'_n}{\Delta' \text{ is a type}}$$

if for each $i$, $1 \leq i \leq n$, $\dfrac{x_1 : \Delta_1, \ldots, x_{i-1} : \Delta_{i-1}}{\Delta_i = \Delta'_i}$ and $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta = \Delta'}$.

We define the relation $\equiv$:

- between the $T$-rules of a theory $U$ by

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta_{n+1} \text{ is a type}} \equiv \frac{y_1 : \Omega_1, \ldots, y_m : \Omega_m}{\Omega_{m+1} \text{ is a type}}$$

if, and only if, $m = n$ and for each $1 \leq i \leq n+1$

$$\frac{x_1 : \Delta_1, \ldots, x_{i-1} : \Delta_{i-1}}{\Delta_i = \Omega_i[x_1|y_1, \ldots, x_{i-1}|y_{i-1}]}$$

is a derived rule of $U$.

- between $\in$-rules by

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{t : \Delta} \equiv \frac{y_1 : \Omega_1, \ldots, y_m : \Omega_m}{s : \Omega}$$

if, and only if,

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta \text{ is a type}} \equiv \frac{y_1 : \Omega_1, \ldots, y_m : \Omega_m}{\Omega \text{ is a type}}$$

and $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{t = s[x_1|y_1, \ldots, x_n|y_n] : \Delta}$ is a derived rule of $U$.

## 1.4 Contexts and Realizations

In this section we will introduce context categories, a powerful interpretation of general algebraic theories that will be key later on. We will start by defining the category of contexts and realizations of a theory $U$.

**Definition 1.17.** *If $\langle x_1 : \Delta_1, \ldots, x_n : \Delta_n \rangle$ and $\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle$ are contexts of $U$, then a **realization** of $\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle$ with regard of $\langle x_1 : \Delta_1, \ldots, x_n : \Delta_n \rangle$ is a m-tuple $\langle t_1, \ldots, t_m \rangle$ such that for each $j$, $1 \leq j \leq m$, the rule*

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{t_j : \Omega_j[t_1|y_1, \ldots, t_{j-1}|y_{j-1}]}$$

*is a derived rule of $U$.*

Then, for a theory $U$, we define the category of contexts and realizations of $U$, $R(U)$, as the category whose objects are contexts of $U$, and where we have a morphism

$$\langle t_1, \ldots, t_m \rangle : \langle x_1 : \Delta_1, \ldots, x_n : \Delta_n \rangle \longrightarrow \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle$$

for each realization $\langle t_1, \ldots, t_m \rangle$ of $\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle$ with regard of $\langle x_1 : \Delta_1, \ldots, x_n : \Delta_n \rangle$. The identity of an object $\langle x_1 : \Delta_1, \ldots, x_n : \Delta_n \rangle$ is the realization $\langle x_1, \ldots, x_n \rangle$. And the composition of two morphisms

$$\langle t_1, \ldots, t_m \rangle : \langle x_1 : \Delta_1, \ldots, x_n : \Delta_n \rangle \longrightarrow \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle$$

$$\langle s_1, \ldots, s_w \rangle : \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle \longrightarrow \langle z_1 : \Lambda_1, \ldots, z_w : \lambda_w \rangle$$

is defined as

$$\langle s_1, \ldots, s_w \rangle \circ \langle t_1, \ldots, t_m \rangle = \langle s_1[t_1|y_1, \ldots, t_m|y_m], \ldots, s_w[t_1|y_1, \ldots, t_m|y_m] \rangle$$

The set of objects of $R(U)$ i.e. the set of contexts of $U$ is structured as a tree with a single least element $\langle\rangle$, the empty context, and where every context $\langle x_1 : \Delta_1, \ldots, x_n : \Delta_n$ is preceded by $x_1 : \Delta_1, \ldots, x_{n-1} : \Delta_{n-1}$. Moreover, for any context $\langle x_1 : \Delta_1, \ldots, x_n : \Delta_n, x_{n+1} : \Delta_{n+1}, \ldots, x_{n+m} : \Delta_{n+m} \rangle$ of $U$, $\langle x_1, \ldots, x_n \rangle$ is a realization of $\langle x_1 : \Delta_1, \ldots, x_n : \Delta_n \rangle$ with respect to $\langle x_1 : \Delta_1, \ldots, x_{n+m} : \Delta_{n+m} \rangle$. We name that morphism as $p(\langle x_1 : \Delta_1, \ldots, x_{n+m} : \Delta_{n+m} \rangle, \langle x_1 : \Delta_1, \ldots, x_n : \Delta_n \rangle)$.

**Lemma 1.18.** *If $A = \langle x_1 : \Delta_1, \ldots, x_n : \Delta_n \rangle$, $A' = \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle$ and $B = \langle y_1 : \Omega_1, \ldots, y_{m+w} : \Omega_{m+w}$ are contexts, and $f = \langle t_1, \ldots, t_m \rangle$ is a realization of $\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle$ with regard to $\langle x_1 : \Delta_1, \ldots, x_n : \Delta_n \rangle$, then*

$$\begin{tikzcd} f^*B \arrow[r, "q(f,B)"] \arrow[d, "p(f^*B,A)"'] & B \arrow[d, "p(B,A')"] \\ A \arrow[r, "f"'] & A' \end{tikzcd}$$

*is a pullback diagram in $R(U)$, where*

$$f^*B = \langle x_1 : \Delta_1, \ldots, x_n : \Delta_n, y_{m+1} : \Omega_{m+1}[t_1|y_1, \ldots, t_m|y_m], \\ \ldots, y_{m+w} : \Omega_{m+w}[t_1|y_1, \ldots, t_m|y_m] \rangle$$

$$q(f, B) = \langle t_1, \ldots, t_m, y_{m+1}, \ldots, y_{m+w} \rangle$$

This properties motivate the following definition:

**Definition 1.19.** *A **contextual category** consists of*

1. *a category $C$ with a terminal object $1$;*
2. *a tree structure on the objects of $C$ such that the terminal object $1$ is the unique least element of the tree;*
3. *for all $A, B \in ObC$ such that $A \lhd B$ a morphism $p(B) : B \to A$ in $C$ and that will be written as $B \twoheadrightarrow A$;*

4. *for all $A, A' \in ObC$ and for all $f : A \to A'$ in $C$, for all $B \in ObC$ such that $A' \lhd B$, an object $f^*B$ in $C$ and a morphism $q(f, B) : f^*B \to B$ such that the diagram*

$$\begin{tikzcd} f^*B \arrow[r, "q(f,B)"] \arrow[d] & B \arrow[d] \\ A \arrow[r, "f"'] & A' \end{tikzcd}$$

*is a pullback in $C$ such that*

   (a) *for all $A, B \in ObC$ such that $A \lhd B$, $id_A^*B = B$ and $q(id_A, B) = id_B$;*

   (b) *when*

$$\begin{tikzcd} & & B \arrow[d] \\ A \arrow[r, "f"] & A' \arrow[r, "f'"] & A'' \end{tikzcd}$$

   *in $C$, then $(f' \circ f)^*B = f^*(f'^*B)$ and $q(f' \circ f, B) = q(f', B) \circ q(f, f'^*B)$.*

The category $R(U)$ is structured as a contextual category by the tree structure and pullbacks defined earlier.

The contextual category $R(U)$ has an homomorphic image $C(U)$ induced by the equivalence relation of intended equivalence of denotation. More precisely, $C(U)$ is the category of equivalence classes of contexts and realizations under the following relation:

- Among contexts:

$$\langle x_1 : \Delta_1, \ldots, x_n : \Delta_n \rangle \equiv \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle$$

  if, and only if,

$$\frac{x_1 : \Delta_1, \ldots, x_{n-1} : \Delta_{n-1}}{\Delta_n \text{ is a type}} \equiv \frac{y_1 : \Omega_1, \ldots, y_{m-1} : \Omega_{m-1}}{\Omega_m \text{ is a type}}$$

- If we have two realizations $\lambda t_1, \ldots, t_m \rangle$ of $\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle$ with regard to $\langle x_1 : \Delta_1, \ldots, x_n : \Delta_n \rangle$, and $\lambda s_1, \ldots, s_m \rangle$ of $\langle y'_1 : \Omega'_1, \ldots, y'_m : \Omega'_m \rangle$ with regard to $\langle x'_1 : \Delta'_1, \ldots, x'_n : \Omega'_n \rangle$, we say that $\langle t_1, \ldots, t_m \rangle \equiv \langle s_1, \ldots, s_m \rangle$ if, and only if, for each $1 \leq j \leq m$,

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{t_j : \Omega_j[t_1|y_1, \ldots, t_{j-1}|y_{j-1}]} \equiv \frac{x'_1 : \Delta'_1, \ldots, x'_n : \Delta'_n}{s_j : \Omega'_j[s_1|y'_1, \ldots, s_{j-1}|y'_{j-1}]}$$

We will define *Fam*, a category that plays the same role among contextual categories as *Set* among categories. The tree of objects of *Fam* is the tree of sets, families of sets, families of families of sets and so on, with a formally adjoined least element 1. For $n, m \geq 0$, if $1 \lhd A_1 \lhd \cdots \lhd A_n$ and $1 \lhd B_1 \lhd \cdots \lhd B_m$ in *Fam*, then the set of morphisms of $A_n$ into $B_n$, is the set of $m$-tuples $\langle f_1, \ldots, f_m \rangle$ where $f_1, \ldots, f_m$ are operators such that for each $1 \leq j \leq m$, for $a_1 : A_1, \ldots, a_n : A_n(a_1, \ldots, a_{n-1})$, $f_j(a_1, \ldots, a_n) \in B_j(f_1(a_1, \ldots, a_n), \ldots, f_{j-1}(a_1, \ldots, a_n))$. For $n = 0$, $Hom_{Fam}(1, B_m) = \{\langle b_1, \ldots, b_m \rangle | b_1 \in B_1, \ldots, b_m \in B_m(b_1, \ldots, b_{m-1})\}$. And for $m = 0$, $Hom_{Fam}(A_n, 1) = \{\langle\rangle\}$, hence 1 is the terminal object of *Fam*. To define composition in *Fam* let $1 \lhd A_1 \lhd \cdots \lhd A_n$, $1 \lhd B_1 \lhd \cdots \lhd B_m$, $1 \lhd C_1 \lhd \cdots \lhd C_w$, $\langle f_1, \ldots, f_m \rangle : A_n \to B_m$ and $\langle g_1, \ldots, g_w \rangle : B_m \to C_w$, then $\langle g_1, \ldots, g_w \rangle \circ \langle f_1, \ldots, f_m \rangle = \langle h_1, \ldots, h_w \rangle$ where for all $1 \leq k \leq w$, $h_k$ is defined as

$$h_k(a_1, \ldots, a_n) = g_k(f_1(a_1, \ldots, a_n), \ldots, f_m(a_1, \ldots, a_n))$$

And to define the projections, we consider $1 \lhd A_1 \lhd \cdots \lhd A_n \lhd A$ in *Fam*, then we consider $p(A) : A \to A_n$ in *Fam* given by

$$p(A) = \langle h_1, \ldots, h_n \rangle; \quad h_i(a_1, \ldots, a_n, a) = a_i$$

Finally, we will define the pullbacks. Let $1 \lhd A_1 \lhd \cdots \lhd A_n$, $1 \lhd B_1 \lhd \cdots \lhd B_m \lhd B$ in *Fam* and $\langle f_1, \ldots, f_m \rangle : A_n \to B_m$ in *Fam*, then we define $\langle f_1, \ldots, f_m \rangle^* B$ as

$$\lambda a_1 \in A_1 . \lambda a_2 \in A_2(a_1) \ldots . \lambda a_n \in A_n(a_1, \ldots, a_{n-1}). \\ B(f_1(a_1, \ldots, a_n), \ldots, f_m(a_1, \ldots, a_n))$$

and $q(\langle f_1, \ldots, f_m \rangle, B) = \langle f_1, \ldots, f_m, \gamma \rangle$ where $\gamma$ is the operator defined as $\gamma(a_1, \ldots, a_n, b) = b$.

Homomorphisms between contextual categories are called contextual functors:

**Definition 1.20.** *If $C$ and $C'$ are contextual categories, then a functor $F : C \to C'$ is a* ***contextual functor*** *if*

1. *$F(1) = 1$ and if $A \lhd B$ in $C$, then $F(A) \lhd F(B)$ in $C'$;*
2. *for all $A \in ObC$, $F(p(A)) = p(F(A))$;*
3. *for all $f$ and $B$ such that $f^*B$ is defined in $C$, $F(f^*B) = F(f)^*F(B)$ and $F(q(f, B)) = q(F(f), F(B))$.*

The category of contextual category and contextual functors is named *Con*.

Now we will introduce some final definitions and results regarding contextual categories. Consider $A \leq B$ in a contextual category $C$, then we define $p(B, A)$ :

$B \to A$ in $C$, written as $B \twoheadrightarrow A$, by $p(B, A) = p(X_1) \circ \cdots \circ p(X_n) \circ p(B)$ where $A \lhd X_1 \lhd \cdots \lhd X_n \lhd B$ in $C$. Analogously, whenever $A' \leq B$ in $C$ and $f : A \to A'$ in $C$, we have the following pullback:

$$\begin{tikzcd}
q(\ldots q(f, X_1) \ldots, X_n)^* B \arrow[r, "q(q(\ldots q(f,X_1)\ldots,X_n),B)"] \arrow[d] & B \arrow[d] \\
A \arrow[r, "f"'] & A'
\end{tikzcd}$$

We can write it using the $^*$ notation for shorthand as:

$$\begin{tikzcd}
f^* B \arrow[r, "q(f,B)"] \arrow[d, "p(f^*B,A)"'] & B \arrow[d, "p(B,A)"] \\
A \arrow[r, "f"'] & A'
\end{tikzcd}$$

**Lemma 1.21.** *If $f : A \to A'$ and $A' \leq X \leq C$ in the contextual category $C$, then $f^*B = q(f, X)^*B$ and $q(f, B) = q(q(f, X), B)$, .i.e the following diagram commutes:*

$$\begin{tikzcd}
q(f, X)^* B \arrow[r, "q(q(f,X),B)"] \arrow[d] & B \arrow[d] \\
f^* X \arrow[r, "q(f,X)"] \arrow[d] & X \arrow[d] \\
A \arrow[r, "f"] & A'
\end{tikzcd}$$

For any contextual category $C$ and for $A$ an object of $C$ we can define a contextual category $C_A$ (similar to $C/A$), whose tree of objects consists of the tree of those objects which appear above $A$ in $C$. More precisely, $ObC_A = \{B \in ObC | A \leq B\}$ and for $B, B' \in ObC_A$ we define $Hom_{C_A}(B, B')$ as the set of morphisms $g : B \to B'$ in $C$ such that the following diagram commutes:

$$\begin{tikzcd}
B \arrow[r, "f"] \arrow[dr] & B \arrow[d] \\
& A
\end{tikzcd}$$

If we have a morphism Â·$f : A \to A'$ in $C$, we define the contextual functor $C_f : C_{A'} \to C_A$ that maps an object $B$ of $C_{A'}$ to $f^*B$ in $C_A$ and a morphism $g : B \to B'$

to $f^*g$, the unique morphism from $f^*B$ to $f^*B'$ in $C_A$ such that $q(f, B') \circ f^*g = g \circ q(f, B)$. By condition 4a of the definition 1.19 we have that if $A \in ObC$, then $C_{id_A} = id_{C_A}$. and by condition 4b we have that if $f : A \to A'$ and $f' : A' \to A''$ in $C$, then $C_{f'} \circ C_f = C_{f' \circ f}$. Hence we have a functor $C_- : C^{op} \to Con$.

**Definition 1.22.** *If $F : C \to C'$ is a contextual functor, then if $A \in ObC$ let $F_A : C_A \to C'_{F(A)}$ be the restriction of $F$ to $C_A$. Then $F_A$ is a contextual functor and $F_- : C_- \Rightarrow C'_{F(A)}$ is a natural transformation*

$$\begin{tikzcd}
C^{op} \arrow[dd, "F"'] \arrow[dr, "C_-"] \arrow[dr, phantom, shift right=3ex, "\Downarrow F_-"] & \\
& Con \\
C'^{op} \arrow[ur, "C'_-"'] &
\end{tikzcd}$$

# Chapter 2

# The Proposed Model

Consider the presentation of a theory $U$. The algorithm has one goal, given a derived $T$-rule $\dfrac{Q}{\Delta \text{ is a type}}$ of the theory $U$ it will try to create a term of type $\Delta$ under the context $Q$, i.e. finding a realization from the context $Q$ to the context $(Q, \Delta)$. To do this we will consider the context category $C(U)_Q$, to limit ourselves to reasoning under the context $Q$.

Let $G$ be a finite subcategory of $C(U)_Q$ containing both $Q$ and $(Q, T)$. We apply iteratively the following steps

1. assign a label to all the objects of $G$ in 3 categories: 'keep', 'expand' and 'delete'. (this is where the AI model comes in)

2. Let $G_1$ be the category consisting of all objects of $G$ not labeled as 'delete'.

3. For each object $X$ of $G$ labeled as 'expand' apply the following expansion rules:

   (a)

TODO: intoroducir $\leq$
TODO: operad

# Bibliography

[1] John Cartmell, *Generalized Algebraic Theories and Contextual Categories*, Ph.D. thesis, Oxford University, 1978.

[2] _____, *Generalised algebraic theories and contextual categories*, Annals of Pure and Applied Logic **32** (1986), 209–243.